

Machine Learning vs Deep Learning

Theory

Aspect	Machine Learning (ML)	Deep Learning (DL)
Definition	Algorithms that learn patterns from data using statistical methods	A subset of ML using multi-layered neural networks
Feature Engineering	Manual — you hand-craft features (e.g., extract “edge count” from an image)	Automatic — the network learns features itself, layer by layer
Data Requirement	Works well with small-to-medium datasets (hundreds to thousands of rows)	Needs large datasets (thousands to millions of samples) to shine
Compute	Runs fine on CPU	Usually needs GPU/TPU for practical training speed
Interpretability	More interpretable (e.g., decision trees, linear regression coefficients)	Largely a “black box” — hard to explain why a specific weight matters
Typical Algorithms	Linear/Logistic Regression, Decision Trees, Random Forest, SVM, k-NN, k-Means	CNNs, RNNs/LSTMs, Transformers, GANs, Autoencoders
Training Time	Fast (seconds to minutes typically)	Slow (hours to weeks for large models)
Best For	Structured/tabular data, small data problems	Unstructured data: images, audio, text, video

Key mental model: *All Deep Learning is Machine Learning, but not all Machine Learning is Deep Learning.* DL = ML + (deep, layered neural networks + automatic feature extraction + large data + heavy compute).

The Neuron

A single artificial neuron (perceptron) does three things:

1. **Weighted sum:** $z = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$
2. **Activation:** $a = f(z)$ — squashes/transforms z into the neuron's output
3. **Pass forward:** a becomes an input to the next layer

```
x1 --w1--\  
x2 --w2---> [ Σ (weighted sum) + bias ] --> [ activation f ] --> output  
x3 --w3--/
```

This is directly inspired by biological neurons: dendrites (inputs) → cell body (sum) → axon (output), firing only if a threshold is crossed (activation).

Weights

- Weights are the **learnable parameters** that determine how much influence each input has on a neuron's output.
- A large positive weight → that input strongly pushes the output up.
- A large negative weight → that input strongly pushes the output down.
- A weight near zero → that input is nearly ignored.
- Weights start **randomly initialized** and are updated during training via gradient descent + backpropagation.

Why not initialize weights to zero? If all weights start at 0 (or any identical value), every neuron in a layer computes the exact same output and receives the exact same gradient update — they never differentiate (“symmetry problem”).

Practical rule of thumb: use `He init` for ReLU-based networks (the default in most modern CNNs), and `Xavier init` for Tanh/Sigmoid networks. In PyTorch/Keras this is usually the framework default — but it's good to know what's happening under the hood.

Bias

- Bias is an extra learnable parameter added to the weighted sum **before** activation: $z = w \cdot x + b$.
- It lets the activation function's output be **shifted left/right**, independent of the input values.
- Without bias, every neuron's decision boundary is forced to pass through the origin — massively limiting what the network can represent.

Activation Functions

Without activation functions, stacking layers is pointless — a composition of purely linear functions is still just linear ($w_2(w_1x) = (w_2w_1)x$), so a 100-layer “network” would collapse to the same power as a single layer. **Activation functions introduce non-linearity**, letting networks learn arbitrarily complex functions (curves, boundaries, patterns).

Function	Formula	Range	Typical Use	Pros	Cons
Sigmoid	$1/(1+e^{-z})$	(0, 1)	Binary classification output	Smooth, probabilistic interpretation	Vanishing gradients, not zero-centered
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	(-1, 1)	Hidden layers (older nets)	Zero-centered	Still vanishing gradients
ReLU	$\max(0, z)$	$[0, \infty)$	Hidden layers (default choice)	Fast, avoids vanishing gradient for $z > 0$	“Dying ReLU” (neurons stuck at 0)
Leaky ReLU	$\max(\alpha z, z), \alpha \approx 0.01$	$(-\infty, \infty)$	Hidden layers	Fixes dying ReLU	Extra hyperparameter
Softmax	$e^{z_i} / \sum e^{z_j}$	(0,1), sums to 1	Multi-class output layer	Gives class probabilities	Only for output layer

Practical: Implementing & plotting activation functions

`import numpy as np`

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

```
def sigmoid_derivative(z):  
    s = sigmoid(z)  
    return s * (1 - s)
```

```
def tanh(z):  
    return np.tanh(z)
```

```

def tanh_derivative(z):
    return 1 - np.tanh(z) ** 2

def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return (z > 0).astype(float)

def leaky_relu(z, alpha=0.01):
    return np.where(z > 0, z, alpha * z)

def softmax(z):
    exp_z = np.exp(z - np.max(z)) # subtract max for numerical stability
    return exp_z / np.sum(exp_z)

# Quick test
z = np.array([-2.0, -0.5, 0.0, 0.5, 2.0])
print("Input:      ", z)
print("Sigmoid:    ", np.round(sigmoid(z), 3))
print("Tanh:       ", np.round(tanh(z), 3))
print("ReLU:       ", np.round(relu(z), 3))
print("Leaky ReLU:  ", np.round(leaky_relu(z), 3))
print("Softmax:    ", np.round(softmax(z), 3), " (sums to",
softmax(z).sum(), ")")

```

```

Input:      [-2.  -0.5  0.   0.5  2. ]
Sigmoid:    [0.119 0.378 0.5   0.622 0.881]
Tanh:       [-0.964 -0.462 0.    0.462 0.964]
ReLU:       [0.   0.   0.   0.5  2. ]
Leaky ReLU: [-0.02 -0.005 0.    0.5   2. ]
Softmax:    [0.012 0.05  0.082 0.135 0.72 ] (sums to 1.0)

```

Practical rule of thumb (2025+ default choices): - Hidden layers → **ReLU** (or GELU/Swish for transformer-style architectures) - Binary classification output → **Sigmoid** - Multi-class classification output → **Softmax** - Regression output → **No activation (linear/identity)**

Artificial Neural Networks (ANN)

An ANN is a computing system loosely inspired by biological brains. It's built from layers of interconnected **neurons**:

Input Layer → Hidden Layer(s) → Output Layer
(features) (learned representations) (prediction)

- **Input layer**: one node per feature (e.g., 784 nodes for a 28×28 pixel image).
- **Hidden layer(s)**: where the “learning” happens — each layer transforms the data into a more abstract representation. “Deep” learning simply means *many* hidden layers.
- **Output layer**: produces the final prediction (a class probability, a number, etc.)

Each connection between neurons has a **weight**, each neuron has a **bias**, and after summing inputs, an **activation function** decides what signal to pass forward. This whole pipeline is what you'll build piece-by-piece in the sections below.

Forward Propagation

Forward propagation (forward pass) is the process of pushing input data **through** the network, layer by layer, to produce a prediction:

$$\begin{aligned} a[0] &= x \quad (\text{input}) \\ z[1] &= W[1] \cdot a[0] + b[1] & a[1] &= f(z[1]) \\ z[2] &= W[2] \cdot a[1] + b[2] & a[2] &= f(z[2]) \\ &\dots \\ z[L] &= W[L] \cdot a[L-1] + b[L] & a[L] &= f(z[L]) = \hat{y} \quad (\text{prediction}) \end{aligned}$$

Each layer's output becomes the next layer's input. This is a pure computation — no learning happens here; forward propagation just computes the current prediction given the current weights.

Loss Functions

A loss function measures **how wrong** a prediction is compared to the true label. Training = trying to minimize this number.

Loss Function	Formula	Used For
Mean Squared Error (MSE)	$(1/n)\sum (y - \hat{y})^2$	Regression
Mean Absolute Error (MAE)	$(1/n)\sum y - \hat{y} $	Regression (robust to outliers)
Binary Cross-Entropy	$-[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	Binary classification
Categorical Cross-Entropy	$-\sum y_i \cdot \log(\hat{y}_i)$	Multi-class classification

Why cross-entropy for classification instead of MSE? Cross-entropy's gradient is much steeper when predictions are confidently wrong, so the model learns faster and doesn't suffer from vanishing gradients the way MSE + sigmoid does.

Practical: Implementing loss functions

```
import numpy as np

def mse_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def mae_loss(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

def binary_cross_entropy(y_true, y_pred, eps=1e-12):
    y_pred = np.clip(y_pred, eps, 1 - eps) # avoid log(0)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

def categorical_cross_entropy(y_true, y_pred, eps=1e-12):
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.sum(y_true * np.log(y_pred)) / y_true.shape[0]

# Regression example
y_true_reg = np.array([3.0, 5.0, 2.5])
y_pred_reg = np.array([2.5, 5.5, 2.0])
print("MSE:", mse_loss(y_true_reg, y_pred_reg))
print("MAE:", mae_loss(y_true_reg, y_pred_reg))

# Binary classification example
y_true_bin = np.array([1, 0, 1, 1])
y_pred_bin = np.array([0.9, 0.1, 0.8, 0.4])
print("BCE:", binary_cross_entropy(y_true_bin, y_pred_bin))
```

```
# Multi-class example (one-hot labels)
y_true_cat = np.array([[0, 1, 0], [1, 0, 0]])
y_pred_cat = np.array([[0.1, 0.8, 0.1], [0.7, 0.2, 0.1]])
print("CCE:", categorical_cross_entropy(y_true_cat, y_pred_cat))

MSE: 0.25
MAE: 0.4999999999999999
BCE: 0.32574...
CCE: 0.2560...
```

Gradient Descent

Gradient descent is the optimization algorithm that **updates weights to reduce loss**. The gradient ($\partial \text{Loss} / \partial W$) tells us the direction of steepest *increase* in loss — so we step in the *opposite* direction:

```
W_new = W_old - learning_rate *  $\partial \text{Loss} / \partial W$ 
b_new = b_old - learning_rate *  $\partial \text{Loss} / \partial b$ 
```

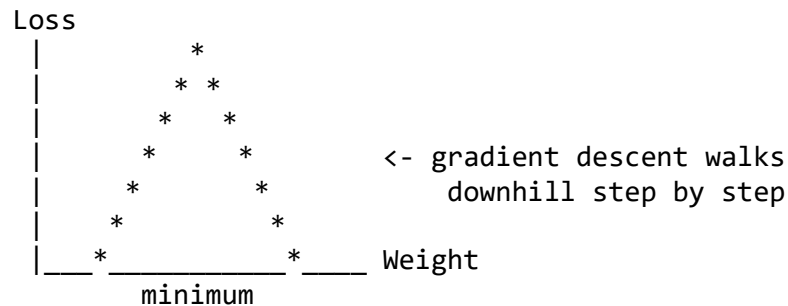
The basic update rule is:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

where:

- W = model parameter
- L = loss
- $\frac{\partial L}{\partial W}$ = gradient
- η = learning rate

- **Learning rate (α):** step size. Too high → overshoots/diverges. Too low → painfully slow convergence.
- **Batch Gradient Descent:** compute gradient using the ENTIRE dataset per step. Stable but slow for big data.
- **Stochastic Gradient Descent (SGD):** compute gradient using ONE sample per step. Fast but noisy.
- **Mini-batch Gradient Descent:** compute gradient using a small batch (e.g., 32–256 samples). The practical default — balances speed and stability.



Backpropagation

Backpropagation is the algorithm that efficiently computes $\partial \text{Loss} / \partial \mathbf{W}$ for every weight in the network, using the **chain rule** of calculus, working backward from the output layer to the input layer.

Why not compute each gradient independently? Because a network can have millions of weights — backprop reuses intermediate computations so the entire gradient computation costs roughly the same as ONE forward pass (instead of millions of separate calculations).

Chain rule intuition for a 2-layer network with loss L :

$$\begin{aligned} \partial L / \partial W_2 &= \partial L / \partial A_2 \cdot \partial A_2 / \partial Z_2 \cdot \partial Z_2 / \partial W_2 \\ \partial L / \partial W_1 &= \partial L / \partial A_2 \cdot \underbrace{\partial A_2 / \partial Z_2 \cdot \partial Z_2 / \partial A_1}_{\text{reused from above}} \cdot \partial A_1 / \partial Z_1 \cdot \partial Z_1 / \partial W_1 \end{aligned}$$

Each layer's gradient is computed by multiplying the "error signal" flowing back from the layer above by the local derivative — that's the entire trick.

Advanced Optimizers

In Ordinary Gradient Descent every update is based **only on the current gradient**.

It has no memory of what happened in previous iterations.

Momentum-based optimizers fix this by giving the optimizer a form of physical inertia — remembering the direction of past updates so it can accelerate through consistent gradient directions and dampen oscillations.

Momentum

Momentum accumulates an exponentially decaying moving average of past gradients (called the velocity, v) and uses that velocity — rather than the raw gradient — to update the weights. It behaves like a ball rolling downhill that gathers speed along consistent slopes and is less affected by small bumps or noisy gradients.

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w_t)$$
$$w_{t+1} = w_t - \eta \cdot v_t$$

β (typically 0.9) controls how much of the past velocity is retained. Higher β means more inertia and smoother, faster convergence along consistent directions, at the cost of possible overshoot near the minimum.

Nesterov Accelerated Gradient (NAG)

NAG improves on plain Momentum with a 'look-ahead' correction. Instead of computing the gradient at the current position, it first takes a step in the direction of the accumulated velocity, then computes the gradient at that look-ahead point. This gives the optimizer a preview of where it is heading, letting it correct course earlier and reduce overshoot compared to standard Momentum.

$$w_look = w_t - \eta \cdot \beta \cdot v_{t-1}$$
$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w_look)$$
$$w_{t+1} = w_t - \eta \cdot v_t$$

AdaGrad (Adaptive Gradient Algorithm)

AdaGrad divides the learning rate for each parameter by the square root of the sum of all its squared past gradients. Parameters that have received large or frequent gradients get their effective learning rate reduced, while parameters with small or infrequent gradients keep a relatively larger learning rate.

$$G_t = G_{t-1} + (\nabla L(w_t))^2$$
$$w_{t+1} = w_t - (\eta / \sqrt{G_t + \epsilon}) \cdot \nabla L(w_t)$$

RMSProp (Root Mean Square Propagation)

RMSProp fixes AdaGrad's vanishing learning-rate problem by replacing the ever-growing sum of squared gradients with an exponentially decaying moving average. This means old gradients are gradually 'forgotten', so the effective learning rate does not shrink to zero, making RMSProp much better suited to non-stationary objectives such as deep network training.

$$S_t = \beta \cdot S_{t-1} + (1 - \beta) \cdot (\nabla L(w_t))^2$$
$$w_{t+1} = w_t - (\eta / \sqrt{S_t + \epsilon}) \cdot \nabla L(w_t)$$

Adam (Adaptive Moment Estimation)

Adam combines the ideas of Momentum and RMSProp: it keeps an exponentially decaying average of past gradients (the first moment, like Momentum's velocity) and an exponentially decaying average of past squared gradients (the second moment, like RMSProp's scaling term). Both moments are bias-corrected to account for their initialization at zero, which matters most during the earliest steps of training. Adam is the most widely used optimizer in modern deep learning because it converges quickly and needs little manual learning-rate tuning.

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla L(w_t)$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (\nabla L(w_t))^2$$

$$\hat{m}_t = m_t / (1 - \beta_1)$$

$$\hat{v}_t = v_t / (1 - \beta_2)$$

$$w_{t+1} = w_t - (\eta / (\sqrt{\hat{v}_t} + \varepsilon)) \cdot \hat{m}_t$$
